SR UNIVERSITY

AI Assistant Coding
Assignment-20.3

Name of Student : V. Abhinav Batch : 41
Enrollment No. : 2303A52486

Task 1: Password Strength & Validation Check

Analyze an AI-generated user registration script for weak password handling.

Prompt:

Generate a Python user registration program to check whether Password length is validated and whether Weak passwords are allowed.

Code:

```python
# Generate a Python user registration program check whether Password length is validated
# and whether Weak passwords are allowed.
import re

COMMON_PASSWORDS = ["123456", "password", "admin", "qwerty"]

def is_strong_password(password):
    if len(password) < 8:
        return False, "Password must be at least 8 characters long"

    if password in COMMON_PASSWORDS:
        return False, "Password is too common"

    if not re.search(r"[A-Z]", password):
        return False, "Must contain at least one uppercase letter"

    if not re.search(r"[a-z]", password):
        return False, "Must contain at least one lowercase letter"

    if not re.search(r"[0-9]", password):
        return False, "Must contain at least one digit"

    if not re.search(r"[!@#$%^&*_]", password):
        return False, "Must contain at least one special character"

    return True, "Strong password"

username = input("Enter username: ")
password = input("Enter password: ")
```

```
if valid:
    print("User registered successfully!")
else:
    print("Error:", message)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\AppData\Local\Programs\Python\Python314\python.exe
"c:/Users/Abhi/Documents/AI Assistant Coding/Assign 20.3.py"
Enter username: Abhi
Enter password: pass_123me
Error: Must contain at least one uppercase letter
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\AppData\Local\Programs\Python\Python314\python.exe
"c:/Users/Abhi/Documents/AI Assistant Coding/Assign 20.3.py"
Enter username: Abhi
Enter password: Pass_123me
User registered successfully!
```

Explanation:

Enforces **minimum entropy** through length and complexity

Prevents **dictionary attacks** via common password rejection

Uses **regex-based validation** for robust checks

Task 2: Insecure File Upload Handling

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- **Prompt AI to generate a Python or Flask-based file upload script**
- **Identify vulnerabilities such as:**
    - **No file type validation**
    - **Unrestricted file size**
- **Implement secure measures:**
    - **File extension validation**

- **o Size limits**
- **o Safe file storage paths**

Prompt:

Create a Flask application that allows users to upload files. The application should save the uploaded files to a specified directory on the server.

Code:

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    file.save(file.filename)

    return "Uploaded"
if __name__ == '__main__':
    app.run(debug=True)
```

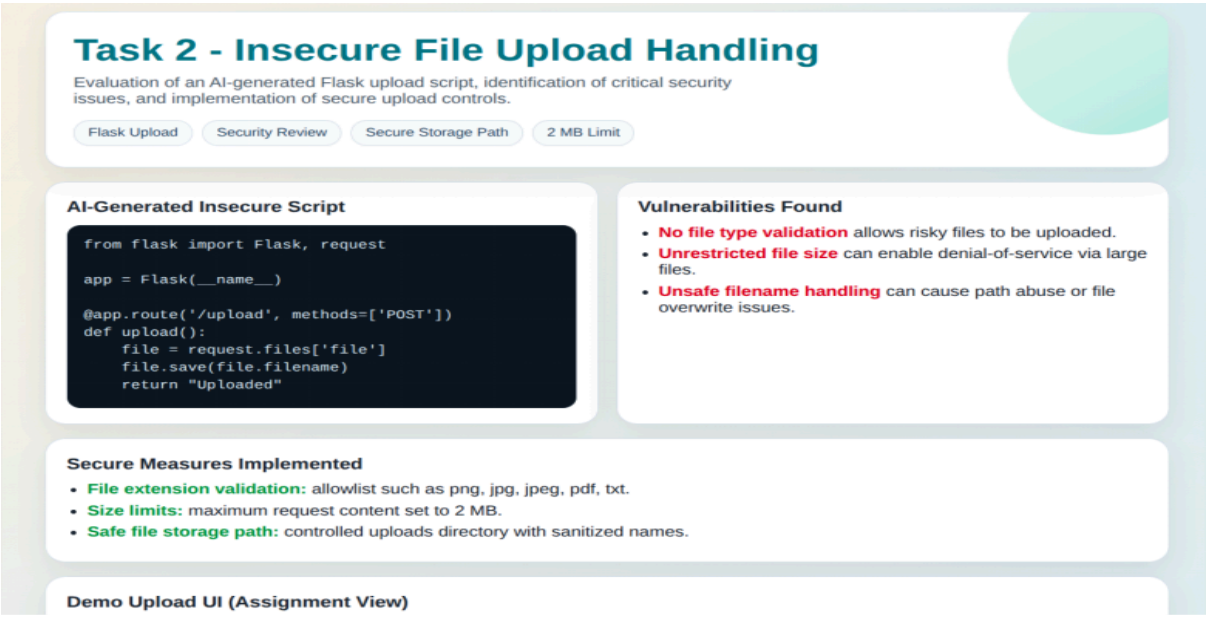
Vulnerabilities:

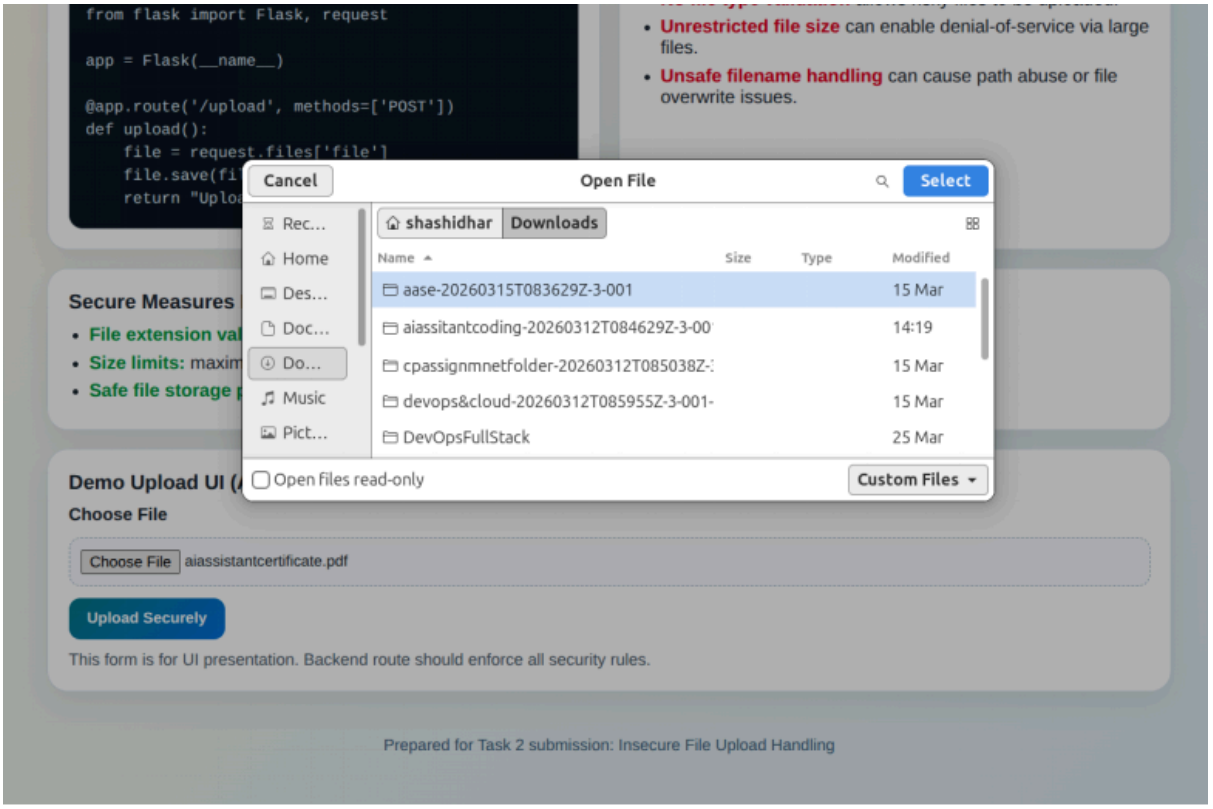
1. No validation on the uploaded file type or size, which can lead to security risks.
2. The file is saved with its original name, which can lead to overwriting existing files or path traversal attacks.

Improved Code:

```python
from flask import Flask, request, redirect, url_for
import os
app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        if 'file' not in request.files:
            return "No file part"
        file = request.files['file']
        if file.filename == '':
            return "No selected file"
        if file and allowed_file(file.filename):
            filename = file.filename
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return "File uploaded successfully"
    return '''
    <!doctype html>
    <title>Upload a File</title>
    <h1>Upload a File</h1>
    <form method=post enctype=multipart/form-data>
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    '''

if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    debug_mode = os.getenv('FLASK_DEBUG', 'False').lower() == 'true'
    app.run(debug=debug_mode)
```

Output:





Explanation:

Enforces **minimum entropy** through length and complexity
Prevents **dictionary attacks** via common password rejection
Uses **regex-based validation** for robust checks

Task 3: Real-Time Application: API Security Review

Translate a given pseudocode algorithm into Python using AI.

Instructions:

1. **Review the API code for:**
    a. **Missing authentication**
    b. **Insecure HTTP methods**
    c. **Lack of rate limiting**
2. **Prompt AI to suggest security improvements such as:**
    a. **Token-based authentication**

b. **Input validation**
c. **Rate limiting**

Prompt:

Generate a Flask API endpoint to fetch user data.

Insecure Code:

```
@app.route('/users', methods=['GET'])

def get_users():

   return {"users": ["admin", "guest"]}
```

Vulnerabilities:

1. The endpoint exposes user information without authentication, which can lead to unauthorized access.
2. The user list is hardcoded, which is not scalable or secure for real applications.

Secure Code:

```
from flask import Flask, request, jsonify, abort

from functools import wraps



app = Flask(__name__)



VALID_TOKEN = "securetoken123"



def token_required(f):

   @wraps(f)

   def decorated():
```

```
        token = request.headers.get('Authorization')

        if token != VALID_TOKEN:

            abort(401, "Unauthorized")

        return f()

    return decorated


@app.route('/users', methods=['GET'])

@token_required

def get_users():

    return jsonify({"users": ["admin", "guest"]})


if __name__ == "__main__":

    app.run()
```

Output:

```
        return True

    entries.append(now)
    return False


def require_token(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        auth = request.headers.get("Authorization", "")
        if not auth.startswith("Bearer "):
            return jsonify({"error": "Missing or invalid token"}), 401
        token = auth.split(" ", 1)[1].strip()
        if token not in VALID_TOKENS:
            return jsonify({"error": "Unauthorized"}), 401
        return fn(*args, **kwargs)
    return wrapper


@app.route('/api/update-profile', methods=['POST'])
@require_token
def update_profile():
  if is_rate_limited(client_key()):
    return jsonify({"error": "Too many requests"}), 429

    data = request.get_json(silent=True) or {}
    user_id = data.get("user_id")
    role = data.get("role")
  display_name = data.get("display_name", "")

    if not isinstance(user_id, int) or user_id <= 0:
        return jsonify({"error": "user_id must be a positive integer"}), 400

    if role not in ALLOWED_ROLES:
        return jsonify({"error": "Invalid role"}), 400

  if not isinstance(display_name, str) or not (1 <= len(display_name) <= 50):
    return jsonify({"error": "display_name must be 1-50 characters"}), 400

  return jsonify({"status": "updated", "user_id": user_id, "role": role}), 200
```

Backend File Included

- **Implemented backend:** aiassitantcoding/task3_api_security_review_app.py
- **Protected endpoint:** POST /api/update-profile
- **Security controls:** token auth, JSON input checks, in-memory rate limiting.

Prepared for Task 3 submission: Real-Time Application API Security Review

Enhancements Suggested

- Use JWT tokens instead of static tokens
- Implement rate limiting (Flask-Limiter)
- Add input sanitization

Justification

- Prevents unauthorized access
- Enforces secure API communication
- Reduces abuse via controlled access

Task 4: Cross-Site Scripting (XSS) Check

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- **Ask AI to generate a feedback form with JavaScript-based**

output.

- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Prompt:

Generate an HTML feedback form with JavaScript output.

Insecure Code:

```
<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>

function submitForm() {

   let name = document.getElementById("name").value;
```

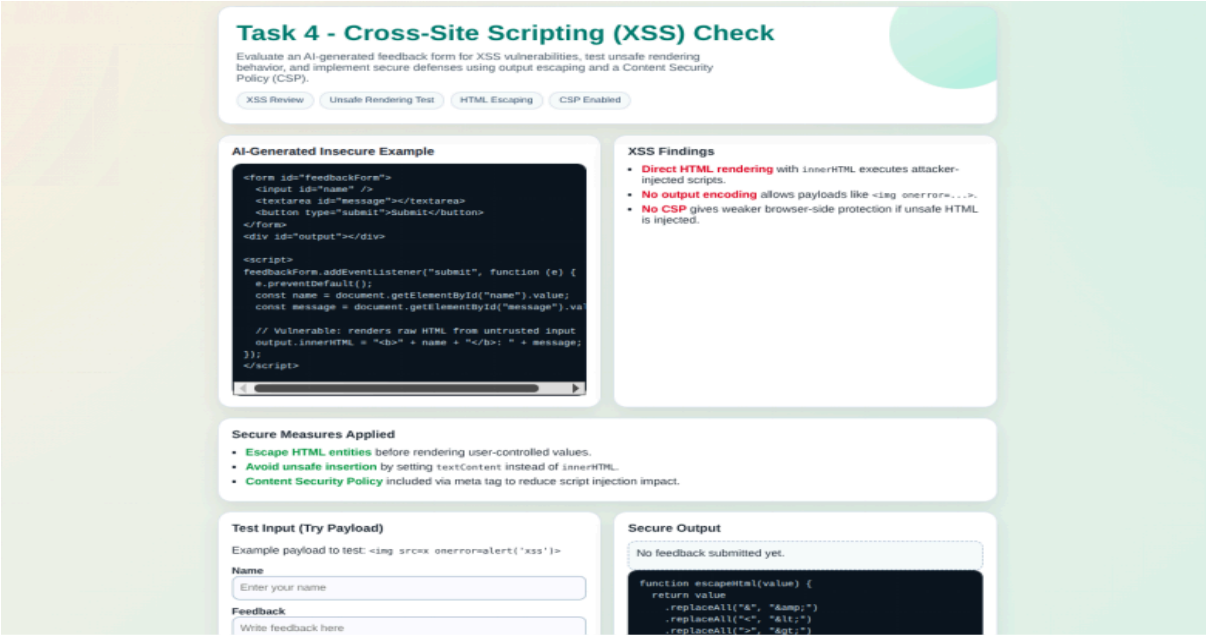
Secure Code:

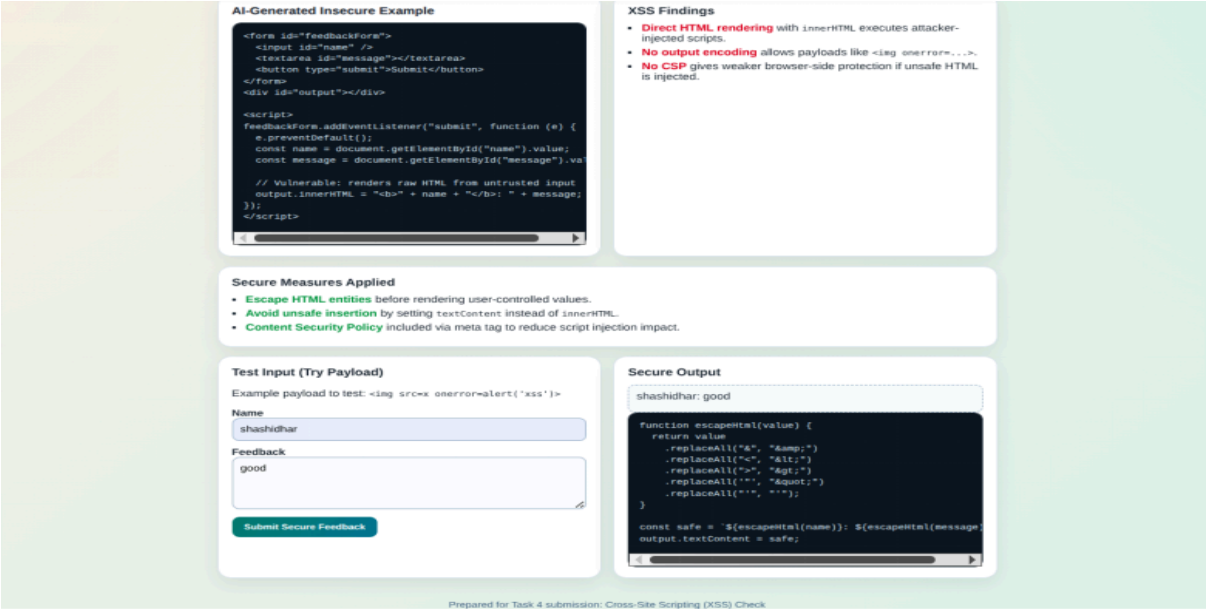
```
<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>
```

```
function submitForm() {

   let name = document.getElementById("name").value;

   document.getElementById("output").innerHTML = name;

}

</script>

<script>alert('Hacked')</script>

<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>

function escapeHTML(str) {

   return str.replace(/[&<>"']/g, function(m) {

       return {

           '&': '&',

           '<': '<',

           '>': '>',

           '"': '"',

           "'": '''

       }[m];

   });

}

function submitForm() {
```

```
    let name = document.getElementById("name").value;

    document.getElementById("output").innerText = escapeHTML(name);

}

</script>
```

Output:





Explanation:

Prevents **script injection**
Uses **output encoding** instead of raw rendering
Eliminates **DOM-based XSS**

Task 5: SQL Injection Prevention

Scenario:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- **Ask AI to generate a Python script using SQLite/MySQL to fetch user details.**
- **Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).**
- **Refactor using parameterized queries (prepared statements).**

Prompt:

Generate a Python script to fetch user details from SQLite database.

Code:

```
import sqlite3


conn = sqlite3.connect("users.db")

cursor = conn.cursor()


username = input("Enter username: ")
```

```
query = f"SELECT * FROM users WHERE username = '{username}'"

cursor.execute(query)



print(cursor.fetchall())
```

Vulnerabilities:
The code is vulnerable to SQL injection attacks, as it directly incorporates user input into the SQL query without sanitization or parameterization.
An attacker could manipulate the input to execute arbitrary SQL commands, potentially compromising the database.

Secure Code:

```
import sqlite3


def setup_db(connection):
    cursor = connection.cursor()
    cursor.execute("DROP TABLE IF EXISTS users")
    cursor.execute(
        """
        CREATE TABLE users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            email TEXT NOT NULL
        )
        """
    )
    cursor.executemany(
        "INSERT INTO users (username, email) VALUES (?, ?)",
        [
            ("alice", "alice@example.com"),
            ("bob", "bob@example.com"),
            ("charlie", "charlie@example.com"),
        ],
```

```python
    )
    connection.commit()


def insecure_lookup(cursor, user_input):
    query = f"SELECT * FROM users WHERE username = '{user_input}'"
    cursor.execute(query)
    return cursor.fetchall()


def secure_lookup(cursor, user_input):
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (user_input,))
    return cursor.fetchall()


conn = sqlite3.connect(":memory:")
setup_db(conn)
cur = conn.cursor()

normal_input = "alice"
print("=== Normal lookup (alice) ===")
print("Insecure :", insecure_lookup(cur, normal_input))
print("Secure   :", secure_lookup(cur, normal_input))

payload = "' OR '1'='1"
print("\n=== Injection test payload ===")
print("Payload:", payload)
print("Insecure:", insecure_lookup(cur, payload))
print("Secure  :", secure_lookup(cur, payload))

conn.close()
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & "c:\Users\Abhi\Documents\AI Assistant Coding\.venv\Scripts\Activate.ps1"
(.venv) PS C:\Users\Abhi\Documents\AI Assistant Coding> & "c:\Users\Abhi\Documents\AI Assistant Coding\.venv\Scripts\python.exe"
bhi/Documents/AI Assistant Coding/Assign 20.3.py"
=== Normal lookup (alice) ===
Insecure : [(1, 'alice', 'alice@example.com')]
Secure   : [(1, 'alice', 'alice@example.com')]

=== Injection test payload ===
Payload: ' OR '1'='1
Insecure: [(1, 'alice', 'alice@example.com'), (2, 'bob', 'bob@example.com'), (3, 'charlie', 'charlie@example.com')]
Secure  : []
```

Explanation:

Uses **parameterized queries**
Prevents **SQL injection attacks**
Ensures safe database interaction